LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 6 – Inheritance



NAMA: NISA

NIM: 103132400044

KELAS: PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Inheritance merupakan salah satu konsep dasar dalam Pemrograman Berorientasi Objek (Object-Oriented Programming/OOP) yang memungkinkan suatu kelas mewarisi atribut dan method dari kelas lain. Dalam Java, kelas yang mewariskan sifat disebut superclass atau base class, sedangkan kelas yang menerima pewarisan disebut subclass atau derived class. Konsep inheritance digunakan untuk mengurangi duplikasi kode dan mempermudah penggunaan kembali atribut serta method yang telah dibuat. Penerapan inheritance dalam Java menggunakan keyword `extends`, sehingga subclass dapat mewarisi anggota superclass sesuai dengan ketentuan hak aksesnya.

Keyword `super` digunakan untuk mengakses anggota milik superclass, seperti atribut, method, dan konstruktor. Keyword ini membantu subclass memanfaatkan anggota kelas induk tanpa harus menuliskan ulang kode yang sudah tersedia. Selain itu, inheritance berkaitan dengan polymorphism, yaitu kemampuan suatu objek untuk memiliki perilaku yang berbeda. Polymorphism memungkinkan penggunaan method yang sama dengan implementasi yang dapat disesuaikan berdasarkan kelas atau objek yang menggunakannya. Dengan demikian, program menjadi lebih fleksibel dan mudah dikembangkan.

Dalam materi inheritance, terdapat dua konsep penting dalam penggunaan method, yaitu overloading dan overriding. Overloading merupakan penggunaan beberapa method dengan nama yang sama dalam satu kelas, tetapi memiliki jumlah atau tipe parameter yang berbeda. Sementara itu, overriding adalah pendefinisian ulang method milik superclass pada subclass dengan nama dan parameter yang sesuai, sehingga subclass dapat memberikan perilaku yang berbeda. Pemahaman mengenai inheritance, keyword `super`, overloading, dan overriding membantu programmer mengembangkan program Java yang lebih terstruktur, efisien, serta mudah dipelihara.

B. Guided

a. Guided1

A.java

```
package com.nisanisaan.modul06.guided1;


public class A {
    int x;
    int y;


    void tampilxy(){
        System.out.println("Nilai x = " + x + " " + "Nilai y = " + y);
    }
}
```

B.java

```
    package com.nisanisaan.modul06.guided1;


public class B extends A{
    int z;
    int hasil;


    void jumlahxyz(){
        hasil = (x+y+z);


        System.out.println("Jumlah (x+y+z) = " + hasil);
    }
}
```

Demo.java

```
    package com.nisanisaan.modul06.guided1;


public class Demo {
    public static void main(String[] agrs){


        A superClass = new A();
        B subClass = new B();

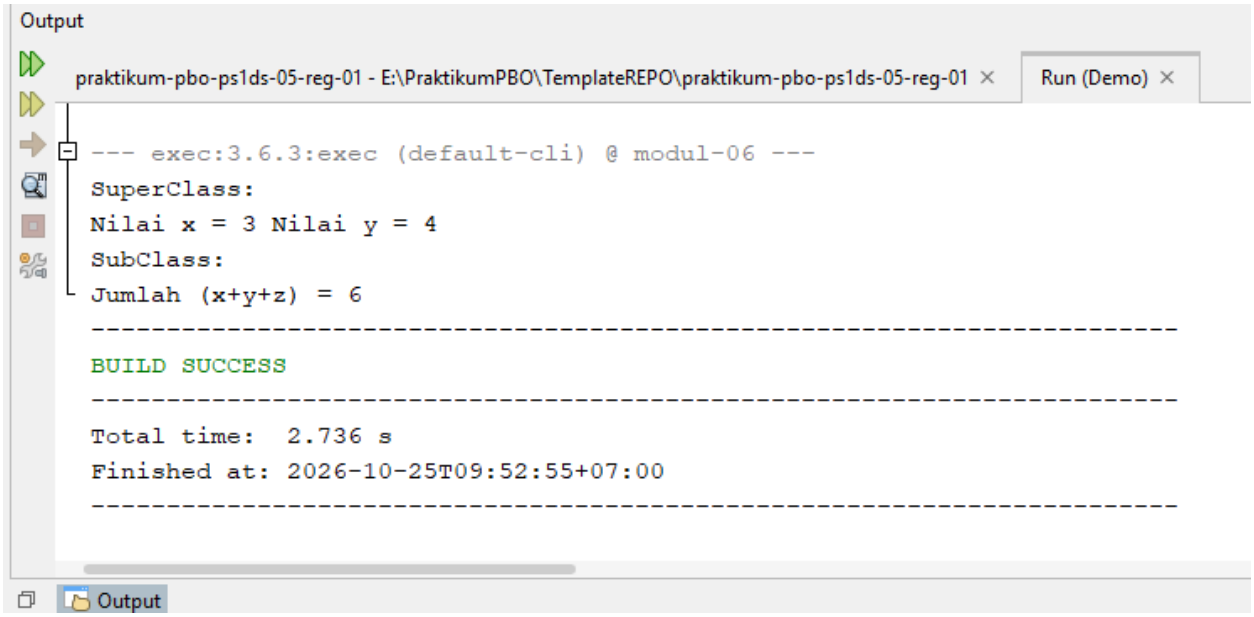

        superClass.x = 3;
        superClass.y = 4;


        System.out.println("SuperClass: ");
        superClass.tampilxy();


        subClass.x = 1;
        subClass.y = 2;
        subClass.z = 3;


        System.out.println("SubClass: ");
        subClass.jumlahxyz();
    }
}
```

Output :



b. Guided2

Employee.java

```
package com.nisanisaan.modul06.guided2;

public class Employee {
    private String nama;
    String departemen;

    public Employee(String nama){
        this.nama = nama;
    }

    public void tampilNama(){
        System.out.println("Nama: " + nama);
    }
}
```

Manager.java

```
package com.nisanisaan.modul06.guided2;

public class Manager extends Employee {
  private String alamat;

  public Manager(String nama, String alamat){
    super(nama);
    this.alamat = alamat;
  }
```

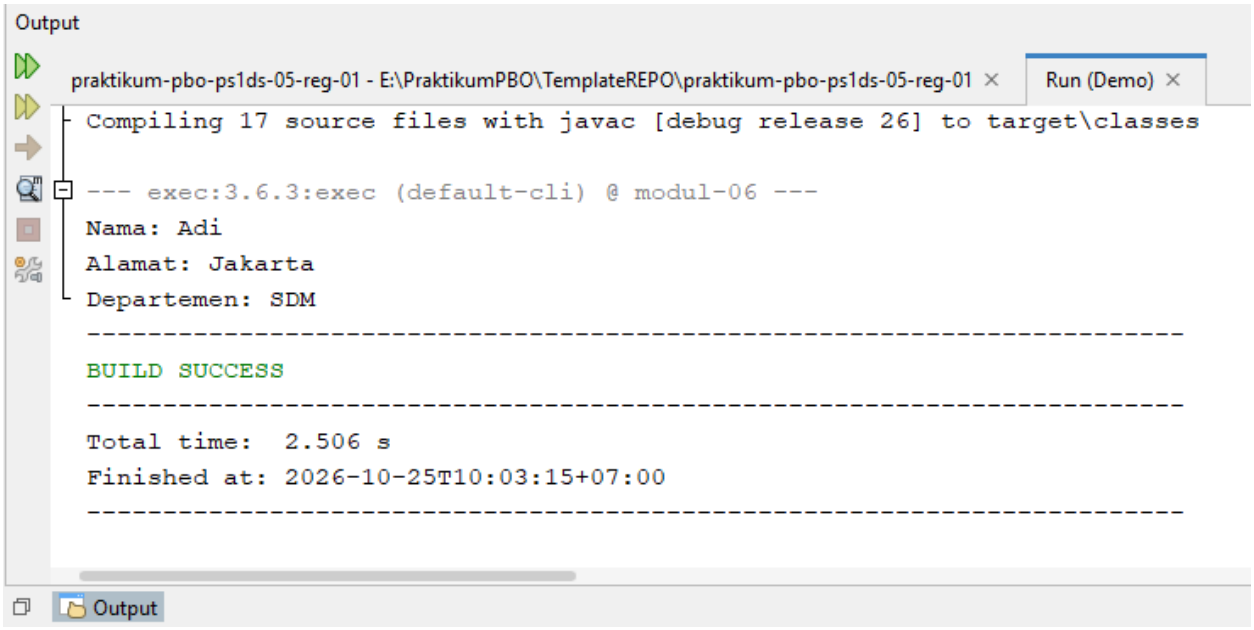
```
  public void tampilInformasi(){
    super.tampilNama();
    System.out.println("Alamat: " + alamat);
    System.out.println("Departemen: " + super.departemen);
  }
}
```

Demo.java

```
package com.nisanisaan.modul06.guided2;

public class Demo {
    public static void main(String[] args) {
        Manager adi = new Manager("Adi", "Jakarta");
        adi.departemen = "SDM";
        adi.tampilInformasi();
    }
}
```

Output :



c. Guided3

Lagu.java

```
package com.nisanisaan.modul06.guided3;

public class Lagu {
    String pencipta;
    String judul;

    void isiParam(String param1){
        judul = param1;
        pencipta = "tidak dikenal";

    }

    void isiParam(String param1, String param2){
        judul = param1;
        pencipta = param2;
    }

    void tampilkan(){
        System.out.println("judul: " + judul + " " + "pencipta: " + pencipta);
    }
}
```

Demo.java

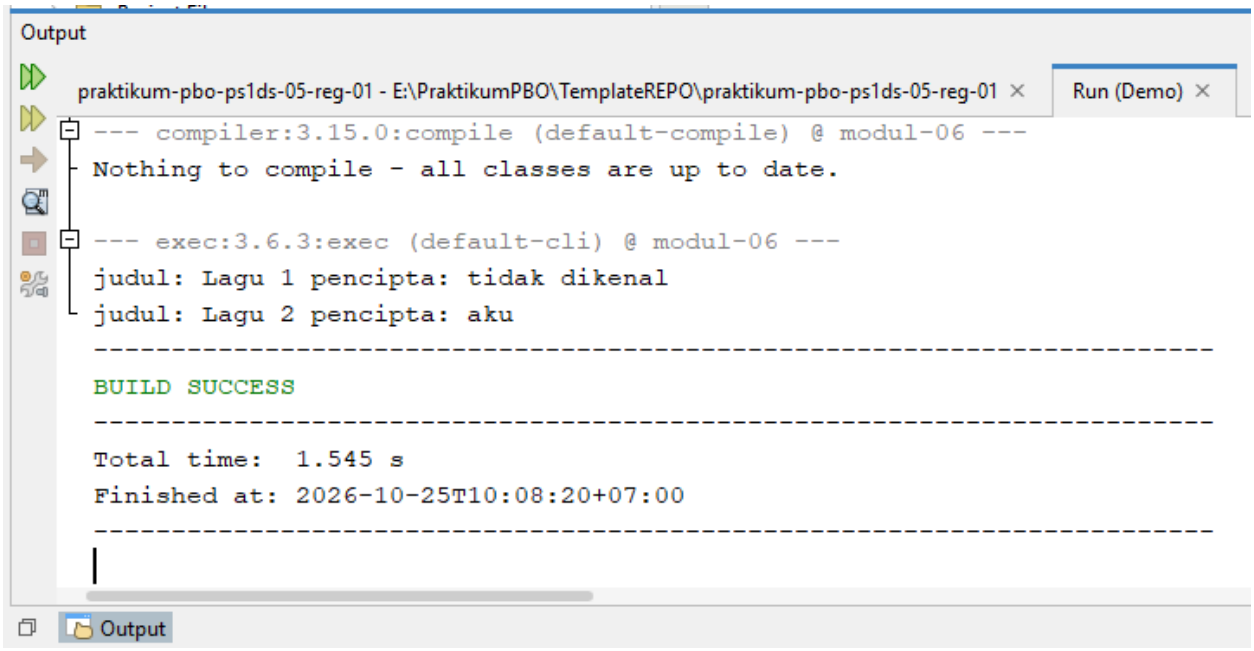
```
package com.nisanisaan.modul06.guided3;

public class Demo {
    public static void main(String[] args) {
        Lagu d,a;
        d = new Lagu();
        a = new Lagu();

        d.isiParam("Lagu 1");
        a.isiParam("Lagu 2", "aku");

        d.tampilkan();
        a.tampilkan();
    }
}
```

Output :



d. Guided4

A.java

```
package com.nisanisaan.modul06.guidede4;

public class A {
    public void tampilkan(){
        System.out.println("Method milik Class A dipanggil...");
    }
}
```

B.java

```
package com.nisanisaan.modul06.guidede4;

public class B extends A {
    public void tampilkan(){
        super.tampilkan();
        System.out.println("Method milik Class B dipanggil...");
    }

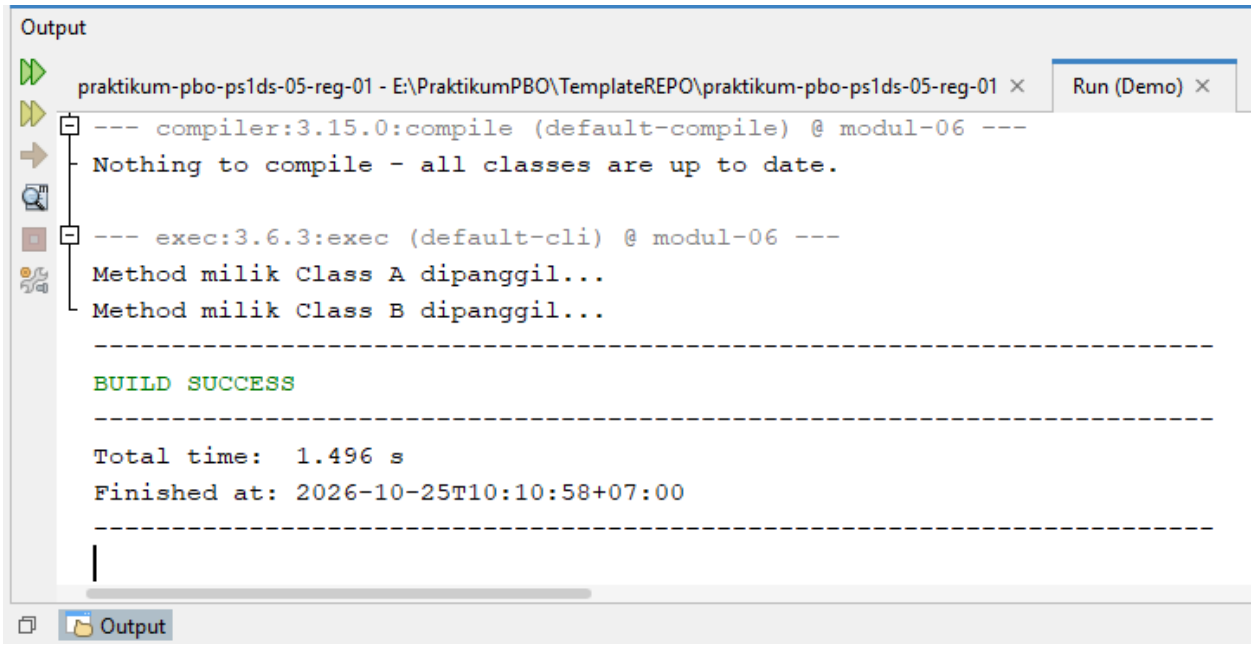
}
```

Demo.java

```
package com.nisanisaan.modul06.guidede4;

public class Demo {
    public static void main(String[] args) {
        B overidB = new B();
        overidB.tampilkan();
    }
}
```

Output :



C. Unguided

1. Soal Unguided

Sebuah tim *machine learning* ingin membuat program Java untuk membandingkan estimasi biaya komputasi dari tiga jenis model: model regresi, model klasifikasi, dan model klasifikasi *deep learning* yang merupakan pengembangan dari model klasifikasi biasa. Penamaan kelas, atribut, dan method ditentukan sendiri, dan semua atribut harus private. Kelas induk menyimpan nama

model dan jumlah data latih, punya *constructor* yang menerima keduanya, cara bagi kelas lain untuk membaca jumlah data latih, method untuk menampilkan informasi, serta dua method penghitung biaya dengan nama sama (*overloading*): tanpa parameter (data latih × 2) dan dengan parameter epoch (data latih × epoch). Kelas turunan pertama (regresi) menambahkan atribut jumlah fitur, memakai *constructor* induk, meng-override method informasi dan biaya tanpa parameter (biaya induk + fitur × 10), serta menambah method biaya baru dengan dua parameter, epoch dan ukuran batch (data latih × epoch ÷ batch), sehingga ada tiga versi penghitung biaya. Kelas turunan kedua (klasifikasi) menambahkan jumlah kelas, meng-override method informasi dan biaya tanpa parameter (biaya induk × jumlah kelas), tanpa meng-override versi epoch. Kelas turunan ketiga (*deep learning*) mewarisi kelas klasifikasi, bukan induk langsung, menambahkan jumlah layer, dan meng-override method informasi serta biaya tanpa parameter (biaya kelas di atasnya + layer × 200). Pada class Main, dibuat tiga objek, yaitu Regresi Linier (500 data, 4 fitur), Decision Tree (800 data, 3 kelas), dan Neural Network (1000 data, 5 kelas, 3 layer), lalu disimpan dalam satu array bertipe kelas induk. Perulangan atas array menampilkan informasi dan biaya tiap model sambil menjumlahkan total biaya, kemudian program menampilkan total biaya serta biaya 10 epoch untuk Neural Network (dari array), biaya 10 epoch untuk Regresi Linier, dan biaya 10 epoch dengan batch 50 untuk Regresi Linier. Hasil yang diharapkan: biaya 1040, 4800, dan 10600, dengan total 16440, lalu 10000, 5000, dan 100.

Jawaban :

ModelML.java

```
package com.nisanisaan.modul06.unguided;


public class ModelML {
    private String namaModel;
    private int jumlahDataLatih;


    public ModelML(String namaModel, int jumlahDataLatih) {
        this.namaModel = namaModel;
        this.jumlahDataLatih = jumlahDataLatih;
    }
```

```
    public int getJumlahDataLatih() {
        return jumlahDataLatih;
    }

    public void tampilkanInfo() {
        System.out.println("Model: " + namaModel + " | Data latih: " + jumlahDataLatih);
    }

    public int hitungBiaya() {
        return jumlahDataLatih * 2;
    }

    public int hitungBiaya(int epoch) {
        return jumlahDataLatih * epoch;
    }
}
```

ModelRegresi.java

```
package com.nisanisaan.modul06.unguided;

public class ModelRegresi extends ModelML {
    private int jumlahFitur;

    public ModelRegresi(String namaModel, int jumlahDataLatih, int jumlahFitur) {
        super(namaModel, jumlahDataLatih);
        this.jumlahFitur = jumlahFitur;
```

```
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah fitur: " + jumlahFitur);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahFitur * 10);
    }

    public int hitungBiaya(int epoch, int ukuranBatch) {
        return getJumlahDataLatih() * epoch / ukuranBatch;
    }
}
```

ModelKlasifikasi.java

```
package com.nisanisaan.modul06.unguided;

public class ModelKlasifikasi extends ModelML {
    private int jumlahKelas;

    public ModelKlasifikasi(String namaModel, int jumlahDataLatih, int jumlahKelas) {
        super(namaModel, jumlahDataLatih);
        this.jumlahKelas = jumlahKelas;
    }
```

```
    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
        System.out.println("Jumlah kelas: " + jumlahKelas);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() * jumlahKelas;
    }
}
```

ModelDeepLearning.java

```
package com.nisanisaan.modul06.unguided;

public class ModelDeepLearning extends ModelKlasifikasi {
    private int jumlahLayer;

    public ModelDeepLearning(String namaModel, int jumlahDataLatih, int jumlahKelas,
int jumlahLayer) {
        super(namaModel, jumlahDataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    @Override
    public void tampilkanInfo() {
        super.tampilkanInfo();
```

```
        System.out.println("Jumlah layer: " + jumlahLayer);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahLayer * 200);
    }
}
```

Main.java

```
package com.nisanisaan.modul06.unguided;

public class Main {
    public static void main(String[] args) {
        ModelRegresi regresi = new ModelRegresi("Regresi Linier", 500, 4);
        ModelML[] daftarModel = {
            regresi,
            new ModelKlasifikasi("Decision Tree", 800, 3),
            new ModelDeepLearning("Neural Network", 1000, 5, 3)
        };

        int totalBiaya = 0;
        for (ModelML m : daftarModel) {
            m.tampilkanInfo();
            int biaya = m.hitungBiaya();
            System.out.println("Biaya komputasi: " + biaya);
            System.out.println("---");
            totalBiaya += biaya;
```

```
        }


        System.out.println("Total biaya: " + totalBiaya);

        System.out.println("Biaya 10 epoch (Neural Network): " +
daftarModel[2].hitungBiaya(10));

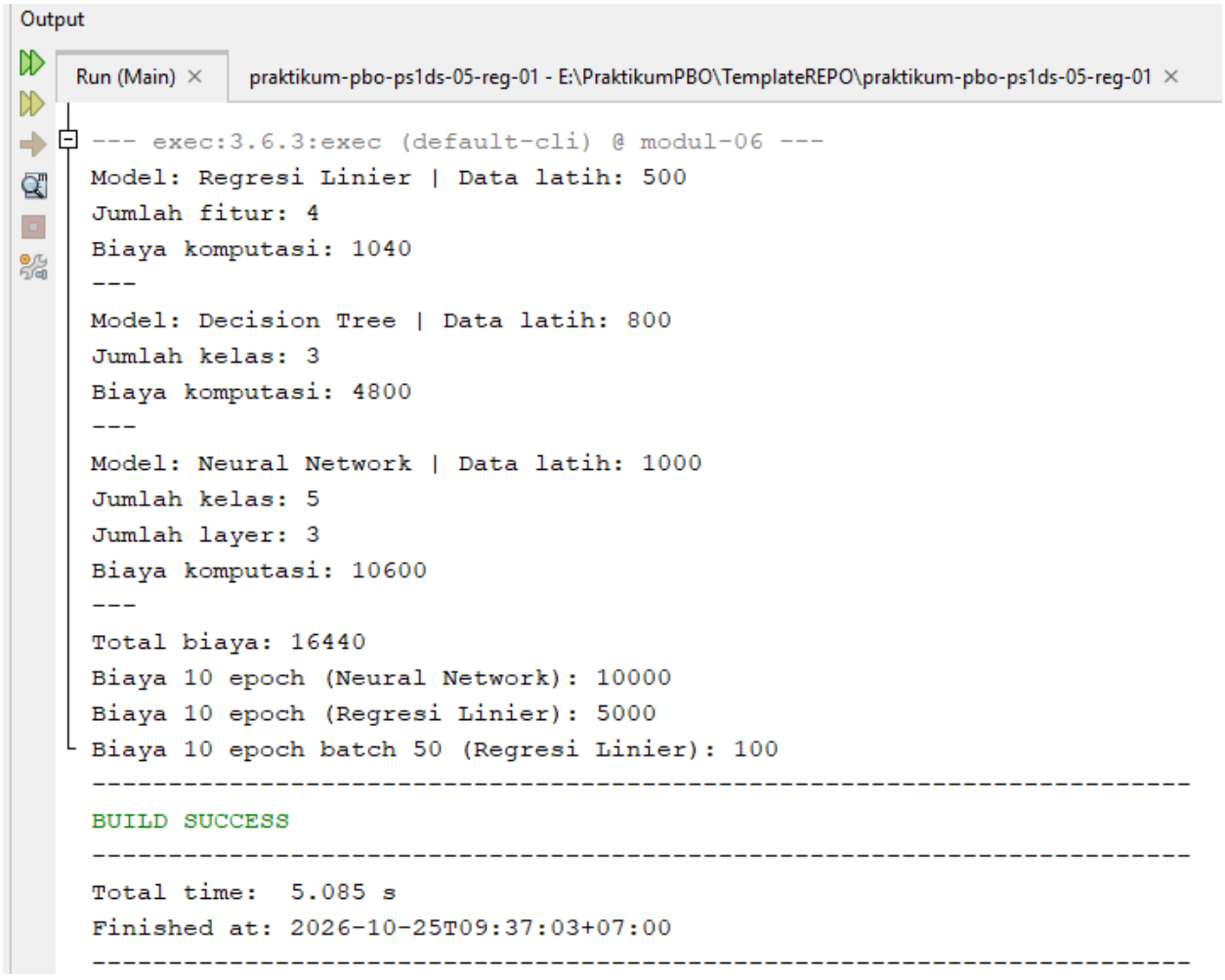
        System.out.println("Biaya 10 epoch (Regresi Linier): " + regresi.hitungBiaya(10));

        System.out.println("Biaya 10 epoch batch 50 (Regresi Linier): " +
regresi.hitungBiaya(10, 50));

    }

}
```

Output :



Penjelasan :

Program ini memodelkan estimasi biaya komputasi untuk tiga jenis model machine learning dengan konsep inheritance. Kelas induk `ModelML` menyimpan nama model dan jumlah data latih, serta punya dua method `hitungBiaya` hasil overloading: tanpa parameter (data latih × 2) dan dengan parameter epoch (data latih × epoch). `ModelRegresi` dan `ModelKlasifikasi` mewarisi `ModelML`, lalu masing-masing meng-override `tampilkanInfo()` dan `hitungBiaya()` dengan memanggil versi induknya lewat `super`, kemudian menambahkan perhitungan sendiri (ditambah fitur × 10 untuk regresi, dikali jumlah kelas untuk klasifikasi). `ModelRegresi` juga punya versi ketiga `hitungBiaya(epoch, ukuranBatch)`. `ModelDeepLearning` mewarisi `ModelKlasifikasi` (pewarisan bertingkat), sehingga biayanya merupakan hasil klasifikasi ditambah layer × 200. Pada `Main`, ketiga objek disimpan dalam satu array bertipe `ModelML`, lalu perulangan memanggil `tampilkanInfo()` dan `hitungBiaya()` pada tiap elemen. Karena polimorfisme, Java otomatis menjalankan versi method sesuai tipe objek aslinya, dan hasilnya dijumlahkan menjadi total biaya. Setelah itu program menampilkan biaya 10 epoch untuk Neural Network (memakai versi induk karena tidak di-override) dan dua biaya untuk Regresi Linier, termasuk versi dengan ukuran batch 50 yang hanya bisa dipanggil lewat variabel bertipe `ModelRegresi`.

D. Kesimpulan

Berdasarkan pembelajaran pada praktikum Modul 6 tentang inheritance, dapat disimpulkan bahwa inheritance merupakan konsep penting dalam Pemrograman Berorientasi Objek (OOP) yang memungkinkan suatu kelas mewarisi atribut dan method dari kelas lain. Melalui praktikum ini, praktikan dapat memahami hubungan antara superclass dan subclass serta penggunaan keyword `extends` untuk menerapkan pewarisan dalam Java. Selain itu, penggunaan keyword `super` membantu mengakses atribut, method, dan konstruktor dari superclass sehingga kode program menjadi lebih efisien dan mengurangi penulisan kode yang berulang.

Praktikum ini juga memberikan pemahaman mengenai overloading dan overriding method sebagai bagian dari konsep penggunaan method dalam Java. Overloading memungkinkan beberapa method memiliki nama yang sama dengan parameter yang berbeda, sedangkan overriding memungkinkan subclass mendefinisikan ulang method dari superclass sesuai kebutuhan. Dengan mempelajari dan mempraktikkan

konsep-konsep tersebut, praktikan dapat meningkatkan pemahaman dalam membuat program yang terstruktur, fleksibel, dan mudah dikembangkan. Oleh karena itu, materi Modul 6 menjadi dasar penting untuk memahami penerapan inheritance dan polymorphism dalam pemrograman berorientasi objek.